UNIVERSIDAD MARIANO GALVEZ DE GUATEMALA

SISTEMAS DE INFORMACION Y CIENCIAS DE LA COMPUTACION



INFORME
PROGRAMACION 2

Eduardo Francisco Rene Cifuentes Chocooj
Carné 0902-25-26342
Franz Johann Rene Cucul Chamam
Carne 0902-25-27771

COBÁN A.V, NOVIEMBRE DE 2026

Introducción

Este proyecto consiste en un Sistema de Gestión de Farmacia desarrollado como aplicación de escritorio en Java Swing, utilizando NetBeans como entorno de desarrollo y SQL Server como base de datos. El sistema organiza información de medicamentos, categorías, proveedores, compras, clientes, recetas médicas, ventas y empleados. Se emplean clases Java y una separación por capas para facilitar el mantenimiento y la consulta de los datos.

El proyecto integra conceptos de Programación Orientada a Objetos, almacenamiento de datos relacional, operaciones de registro y consulta, validación de entradas y manejo de errores. Su finalidad es apoyar las tareas habituales de una farmacia mediante una interfaz gráfica.

Objetivos

Objetivo general

Desarrollar un sistema de escritorio que permita administrar las operaciones principales de una farmacia mediante Java y una base de datos SQL Server.

Objetivos específicos

- Implementar módulos para medicamentos, proveedores, compras, clientes, recetas médicas, ventas e integrantes del personal.
- Representar las entidades del sistema mediante clases y un modelo relacional con claves primarias y foráneas.
- Permitir altas, búsquedas y modificaciones de registros, y consultas mediante tablas e historiales.
- Incorporar validaciones y manejo de excepciones para reducir errores de operación.
- Organizar el código en capas de interfaz, lógica y acceso a datos.

Problema abordado

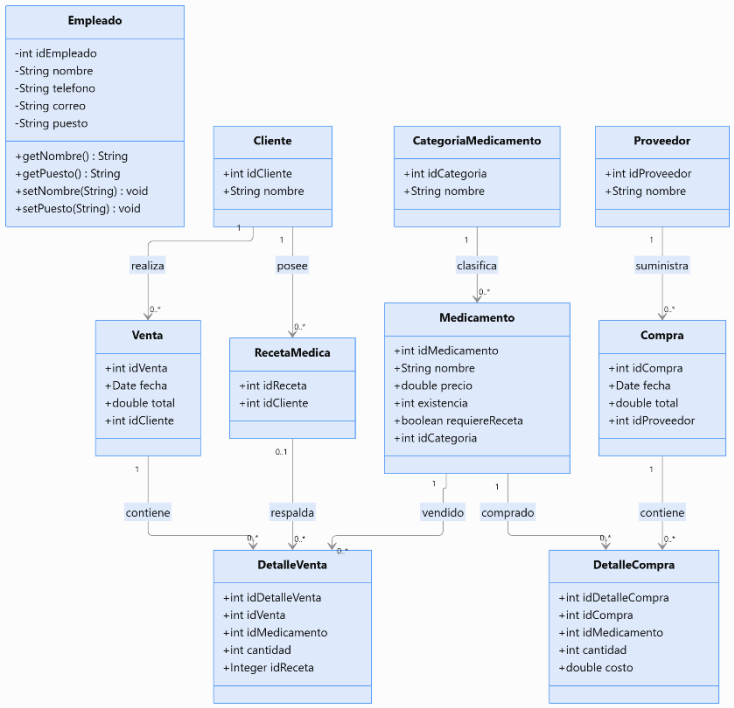
Una farmacia necesita controlar las existencias, precios y categorías de medicamentos, conocer a sus proveedores, registrar compras y ventas y mantener datos de sus clientes. Cuando estos procesos se gestionan por separado, resulta más difícil consultar información actualizada o detectar inconsistencias en el inventario.

El sistema propuesto reúne estos procesos en una sola aplicación. Al registrar una compra se prevé actualizar las existencias; al registrar una venta se comprueba la disponibilidad y se descuenta la cantidad vendida. Además, los medicamentos que requieren receta médica deben quedar relacionados con una receta del cliente.

Diagrama de clases UML

El diagrama de clases representa diez entidades principales: Empleado, Cliente, CategoriaMedicamento, Proveedor, Venta, RecetaMedica, Medicamento, Compra, DetalleVenta y DetalleCompra. Contiene atributos y relaciones; por ejemplo, Cliente se relaciona con Venta y RecetaMedica, mientras que cada Venta contiene registros de DetalleVenta.

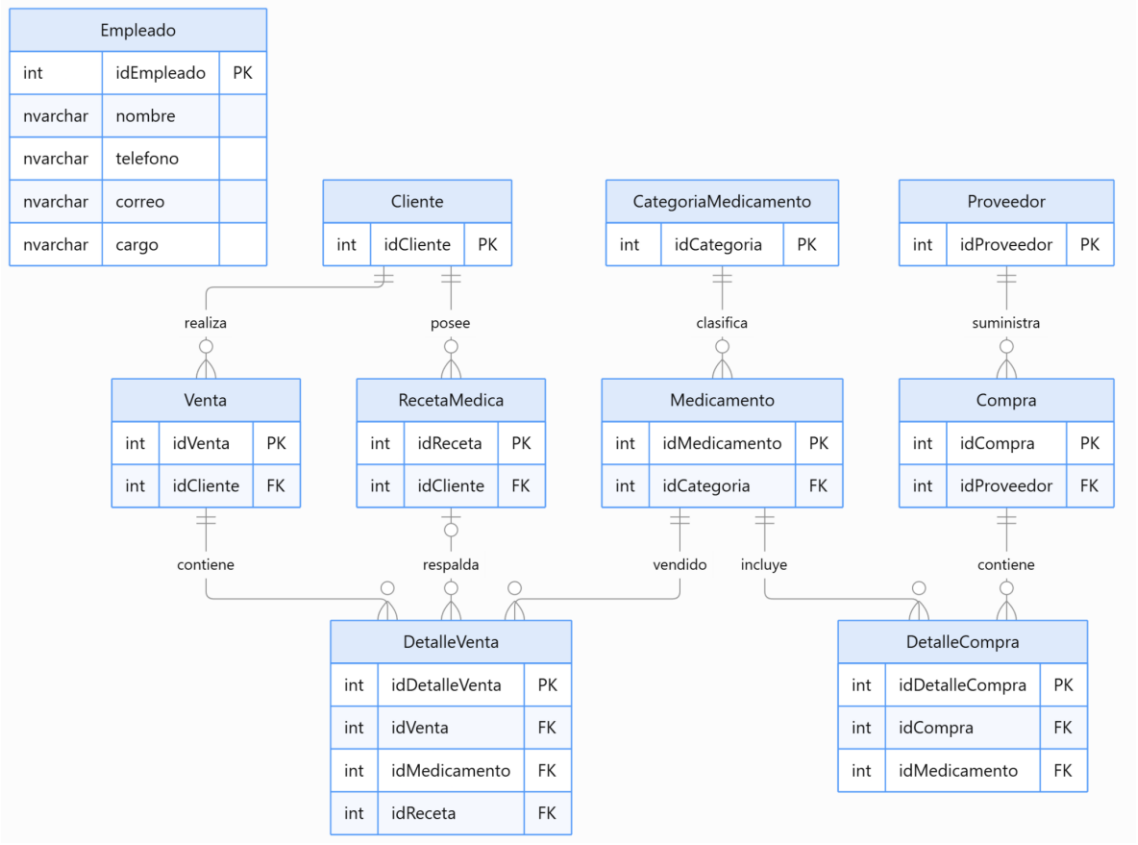
En DetalleVenta se registra la relación entre una venta y sus medicamentos; de forma equivalente, DetalleCompra representa los renglones de una compra. Estas clases muestran el diseño conceptual y su relación con el manejo de datos.



Diseño de base de datos (modelo entidad-relación)

La base de datos GestionFarmacia, utilizada desde Java mediante JDBC, incluye diez tablas principales: CategoriaMedicamento, Cliente, Compra, DetalleCompra, DetalleVenta, Empleado, Medicamento, Proveedor, RecetaMedica y Venta. El script SQL incluido en el proyecto permite verificar sus definiciones y relaciones.

Cada tabla contiene una clave primaria (PK), que identifica sus registros. Las claves foráneas (FK) permiten relacionar entidades; por ejemplo, Venta referencia a Cliente y DetalleVenta relaciona Venta con Medicamento. Las relaciones maestro–detalle también se aplican a Compra y DetalleCompra.



Diseño aplicado: arquitectura por capas

El sistema fue organizado mediante una arquitectura por capas. Esta decisión separa la interfaz de usuario, la lógica de operaciones y el acceso a SQL Server. La carpeta presentacion incluye formularios Swing; modelo contiene entidades; negocio incluye validaciones y operaciones; datos agrupa clases DAO y clases de conexión y registro.

Cuando el usuario registra una venta, la interfaz recoge los datos y utiliza las clases encargadas de procesar la operación; las clases de datos realizan las consultas y actualizaciones necesarias. La separación facilita localizar cada responsabilidad y permite modificar una parte sin mezclarla directamente con el resto.

Ventajas de la arquitectura utilizada

La arquitectura por capas nos permitió mantener una mejor organización del código, ya que cada parte del programa tiene una función específica. Esto facilita encontrar errores y realizar modificaciones sin afectar innecesariamente otros módulos.

Por ejemplo, cuando registramos un medicamento, la capa de presentación recibe la información ingresada por el usuario, la capa de negocio realiza las validaciones necesarias y la capa de datos se encarga de comunicarse con SQL Server para guardar el registro.

Esta separación también facilitó el trabajo en equipo, porque permitió desarrollar diferentes funciones del sistema de forma más ordenada.



Resultados obtenidos

La documentación y los archivos del proyecto muestran una aplicación con pantallas para gestionar medicamentos, proveedores y compras, clientes y recetas médicas, ventas, historial de ventas y empleados. Incluye ventanas Java Swing, clases de entidad, clases de lógica y componentes de consulta SQL.

- Catálogo de medicamentos: registro, edición, eliminación, búsqueda y consulta de existencias, según el manual de usuario.
- Compras: captura de proveedor y renglones de medicamentos; al registrar compras se actualiza el inventario según el flujo documentado.
- Clientes y recetas: asociación de recetas con los clientes correspondientes.
- Ventas: cálculo del total, revisión de recetas cuando corresponde y control de disponibilidad de existencias.
- Historiales: consulta de operaciones registradas mediante tablas de la interfaz.
- Empleados: administración de datos básicos del personal.

El sistema incorpora consultas mediante PreparedStatement para trabajar con la base de datos y reducir el riesgo de inyección SQL. Además, se desarrollaron validaciones para controlar la información ingresada y verificar las existencias de medicamentos antes de registrar una venta.

También se implementaron funciones para administrar los registros de medicamentos, proveedores, clientes y empleados, así como para consultar las compras y ventas realizadas. Estas funciones permiten relacionar los diferentes módulos y mantener organizada la información de la farmacia.

Conclusiones

- El proyecto permitió reunir en una aplicación las operaciones principales de una farmacia, con información organizada por módulos.
- El uso de clases Java y una base de datos relacional ayudó a representar los medicamentos, clientes, compras y ventas y sus relaciones.
- Separar el programa en capas hizo más clara la distribución de responsabilidades y permitió aplicar conocimientos de programación y consultas a bases de datos.